

As you can see, a list of Booleans was returned, representing values that are NaN. Now there are two options of how to deal with the NaN values. First, you can choose to drop all rows with NaN values using the *dropna* function, in the following manner:

```
data.dropna(subset=['Events'])

<class 'pandas.core.frame.DataFrame'>
Int64Index: 162 entries, 1 to 361
Data columns:
EDT                                162 non-null values
Max TemperatureF                   162 non-null values
Mean TemperatureF                  162 non-null values
Min TemperatureF                   162 non-null values
Max Dew PointF                     162 non-null values
MeanDew PointF                     162 non-null values
Min DewpointF                      162 non-null values
Max Humidity                       162 non-null values
Mean Humidity                      162 non-null values
Min Humidity                       162 non-null values
Max Sea Level PressureIn           162 non-null values
Mean Sea Level PressureIn          162 non-null values
Min Sea Level PressureIn           162 non-null values
Max VisibilityMiles                 162 non-null values
Mean VisibilityMiles               162 non-null values
Min VisibilityMiles                162 non-null values
Max Wind SpeedMPH                  162 non-null values
Mean Wind SpeedMPH                162 non-null values
Max Gust SpeedMPH                  162 non-null values
PrecipitationIn                    162 non-null values
CloudCover                         162 non-null values
Events                             162 non-null values
WindDirDegrees                     162 non-null values
dtypes: float64(4), int64(16), object(3)
```

As you can see, there are only 162 rows, which don't contain NaNs in the column 'Events'. The other option you have is to replace the NaN values with something easier to deal with using the *fillna* function. You can do this in the following manner:

```
data["Events"].fillna("")

0
1          Rain
2          Rain
3
4
5  Rain-Thunderstorm
6
7  Fog-Thunderstorm
8          Rain
362
```

```
363
364
365
Name: Events, Length: 366
```

Accessing individual rows

So far we have discussed methods dealing with indexing entire columns, but what if you want to access a specific row in your dataframe? Well, Pandas provides a function called *irow*, which lets you get the value of a specific row. You can use it as follows:

```
data.irow(0)

EDT                                2012-3-10
Max TemperatureF                   56
Mean TemperatureF                  40
Min TemperatureF                   24
Max Dew PointF                     24
MeanDew PointF                     20
Min DewpointF                      16
Max Humidity                       74
Mean Humidity                      50
Min Humidity                       26
Max Sea Level PressureIn           30.53
Mean Sea Level PressureIn          30.45
Min Sea Level PressureIn           30.34
Max VisibilityMiles                 10
Mean VisibilityMiles               10
Min VisibilityMiles                10
Max Wind SpeedMPH                  13
Mean Wind SpeedMPH                 6
Max Gust SpeedMPH                  17
PrecipitationIn                    0.00
CloudCover                         0
Events                             NaN
WindDirDegrees                     138
Name: 0
```



Note: Indices start from 0 for indexing the rows.

Filtering

Sometimes you may need to find rows of special interest to you. Let's suppose we want to find out data points in our data frame, which have a mean temperature greater than 40 and less than 50. You can filter out values from your dataframe using the following syntax:

```
data[(data['Mean TemperatureF']>40) & (data['Mean
TemperatureF']<50)]

<class 'pandas.core.frame.DataFrame'>
```